

# PROJECT REVIEW PREPARATION GUIDE

*Brain Tumor Detection & Classification Using Machine Learning  
(EfficientNetB0 Transfer Learning & Explainable AI)*

|               |                                                                                                                                          |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Presented By: | 1) Ragyari Sruthi (Roll No: 23781A33D5)<br>2) Ratala Venkata Ramakrishna (Roll No: 23781A33D7)<br>3) Shaik Thameem (Roll No: 23781A33F3) |
| Department:   | Dept. of Computer Science & Engineering (AI & ML)                                                                                        |
| Institution:  | Sri Venkateswara College of Engineering and Technology (SVCET),<br>Chittoor                                                              |

## 1. Project Objectives & Overview

The objective of this project is to build an automated, explainable deep learning pipeline to classify brain Magnetic Resonance Imaging (MRI) scans into four distinct categories: Glioma, Meningioma, Pituitary tumor, or Healthy (No Tumor).

This application acts as a clinical support utility that assists radiologists by providing rapid diagnostic validation and explaining model decisions using gradient-based visual localization.

## 2. System Requirements

- **Software:** Python 3.10+, TensorFlow/Keras (model building and training), OpenCV (image processing and Grad-CAM blending), tf2onnx (format conversion), ONNX Runtime (CPU inference execution), Gradio (web front-end).
- **Hardware:** NVIDIA T4 GPU (Google Colab free tier used) for parallel tensor operations during training. Standard CPU (1 Core, 2GB RAM) is sufficient for live cloud inference.

## 3. Dataset Architecture

The dataset utilized is the **Brain Tumor MRI Dataset** by Masoud Nickparvar, programmatically downloaded from Kaggle via the Kaggle API. It consists of **7,023 images** divided as follows:

| Tumor Class | Training Set | Test Set | Total | Clinical Description                           |
|-------------|--------------|----------|-------|------------------------------------------------|
| Glioma      | 1,321        | 300      | 1,621 | Tumor originating in glial cells of the brain. |

|            |       |     |       |                                                 |
|------------|-------|-----|-------|-------------------------------------------------|
| Meningioma | 1,339 | 306 | 1,645 | Tumor in membrane enclosing the brain.          |
| Pituitary  | 1,457 | 300 | 1,757 | Tumor in the hormone-secreting pituitary gland. |
| No Tumor   | 1,595 | 405 | 2,000 | Healthy brain tissue (control set).             |

Input scans are resized to **224 x 224 pixels** with 3 channels (RGB) to match the expected dimensions of the EfficientNet base model.

## 4. Model Design & Training Workflow

We implemented Transfer Learning using the **EfficientNetB0** architecture pre-trained on ImageNet. A custom classification head was appended: Global Average Pooling 2D → Dense Layer (256, ReLU) → Batch Normalization → Dropout (Rate = 0.3) → Dense Layer (4, Softmax).

### Training Strategy:

- **Phase 1: Feature Extraction (15 Epochs, Learning Rate = 1e-3):** The EfficientNetB0 backbone was entirely frozen. Only the added top dense head layers were trained.
- **Phase 2: Fine-Tuning (10 Epochs, Learning Rate = 1e-5):** The top 30 convolutional layers of the EfficientNetB0 backbone were unfrozen and trained at a very low learning rate to adapt features specifically to brain MRI textures without destroying ImageNet weights.

## 5. Critical Engineering Highlight: Double-Rescaling Bug

During training development, a major preprocessing conflict caused the model to experience vanishing gradients, locking weights into predicting the majority class ('No Tumor') for all test images.

**The Conflict:** The training script utilized Keras' ImageDataGenerator(rescale=1./255). However, the EfficientNetB0 network has a built-in rescaling layer as its first layer. This double-rescaling operation divided pixels by 255 twice, shrinking input values to near zero ([0, 0.0039]).

**The Fix:** We removed manual rescaling from the generators and matching preprocessing loops, allowing the raw [0, 255] pixel arrays to reach the model. The model immediately converged, achieving **99.59% accuracy on tumor cases** and **99.99% accuracy on healthy cases**.

## 6. Grad-CAM & Deployment Optimization

- **Grad-CAM Explainability:** Implemented in the training notebook to extract gradient activations from the last convolutional layer (top\_conv) of EfficientNetB0. It maps heatmap overlays directly on MRI tumors to explain model decisions visually, building clinical trust.
- **ONNX Conversion:** Converted the final trained model from Keras (.h5) to ONNX (.onnx) format. We used **ONNX Runtime (ort)** for deployment. The setup shrunk the required runtime packages in the deployment container from 2GB (TensorFlow) to ~15MB, allowing fast inference on simple cloud CPUs in milliseconds.

## 7. Comprehensive Review Q&A; Bank (20 Questions)

Use these questions to study and prepare for direct evaluation from the project review board.

### Q1: What is the primary clinical objective of this project?

**Answer:** The objective is to classify brain MRI scans into four classes: Glioma, Meningioma, Pituitary, and No Tumor (Healthy). It serves as a rapid second-opinion support utility for radiologists to prioritize patient scan queues.

### Q2: Why did you choose Transfer Learning instead of training a Convolutional Neural Network (CNN) from scratch?

**Answer:** Training from scratch requires hundreds of thousands of labeled images to learn general features like edges and textures. Transfer Learning allows us to start with a model (EfficientNetB0) pre-trained on 1.4 million images (ImageNet). We adapt these robust features to our MRI dataset of 7,023 images in a fraction of the time, achieving high accuracy without overfitting.

### Q3: Why did you select EfficientNetB0 over older models like ResNet50 or VGG16?

**Answer:** EfficientNetB0 balances accuracy and resource efficiency through 'compound scaling' (scaling network width, depth, and input resolution uniformly). It has only 5.3 million parameters compared to ResNet50's 25 million and VGG16's 138 million. This reduces training times, minimizes parameters to prevent overfitting, and creates a small model file (~17MB ONNX) that is ideal for lightweight CPU hosting.

### Q4: Where did you get the brain MRI dataset, and how did you load it?

**Answer:** We used the Brain Tumor MRI Dataset by Masoud Nickparvar from Kaggle. We downloaded it programmatically in our training Jupyter notebook using the Kaggle API and securely stored Kaggle credentials in the workspace during download.

### Q5: How is the dataset distributed between training, validation, and testing?

**Answer:** The dataset has 7,023 total images. The training folder has 5,712 images and the test folder has 1,311 images. During training, a 20% validation split was dynamically carved out of the training folder, leaving 4,570 images for active training and 1,142 for validation checks at the end of each epoch.

### Q6: What input shape does the model expect, and why?

**Answer:** The model expects inputs of shape (224, 224, 3) representing a 224x224 pixel image with 3 color channels (RGB). We resized all grayscale MRIs to these dimensions because the pre-trained EfficientNetB0 backbone was originally built for 224x224 ImageNet inputs.

**Q7: What layers did you add to create the custom classification head?**

**Answer:** We appended: 1) a Global Average Pooling 2D layer to flatten the final conv feature maps, 2) a Dense layer with 256 units and ReLU activation to learn task-specific features, 3) a Batch Normalization layer to stabilize training, 4) a Dropout layer with a rate of 0.3 to prevent overfitting, and 5) a final Dense layer with 4 units and Softmax activation to output probabilities for our classes.

**Q8: Why did you use Global Average Pooling 2D (GAP) instead of a Flatten layer?**

**Answer:** A Flatten layer flattens the spatial dimension into a massive vector, leading to a high number of weights in the subsequent Dense layer, which increases overfitting risk. Global Average Pooling averages the spatial activations of each feature map, reducing the output size from 7x7x1280 to a simple 1280-dimensional vector. This massively reduces parameter count and improves regularization.

**Q9: What is the purpose of the Dropout layer, and why was the rate set to 0.3?**

**Answer:** Dropout acts as a regularizer. A rate of 0.3 means that during each training batch, 30% of the nodes in that dense layer are randomly deactivated. This prevents the model from relying too heavily on specific nodes and forces it to learn redundant representations, preventing overfitting on training samples.

**Q10: What is Batch Normalization and what is its role in your model?**

**Answer:** Batch Normalization normalizes the outputs of a layer across each batch. It addresses the internal covariate shift by scaling activations, which stabilizes gradients, speeds up convergence, and acts as a minor regularizer.

## 7. Comprehensive Review Q&A; Bank (Continued)

### Q11: Explain your two-phase training strategy.

**Answer:** In Phase 1, we froze all layers of the EfficientNetB0 backbone and trained only our custom dense head for 15 epochs at a learning rate of  $1e-3$ . In Phase 2, we unfroze the top 30 convolutional layers of the backbone and trained the whole model for 10 epochs at a low learning rate of  $1e-5$  to adjust backbone parameters to MRI-specific structures.

### Q12: Why did you use a lower learning rate ( $1e-5$ ) during Phase 2 (fine-tuning) than Phase 1 ( $1e-3$ )?

**Answer:** The pre-trained backbone already possesses highly valuable weights learned from millions of images. A high learning rate in Phase 2 would trigger large gradient steps that overwrite and corrupt these pre-trained weights. A tiny learning rate ( $1e-5$ ) gently adjusts the top conv layers so they adapt to MRI scans without destroying pre-learned features.

### Q13: Describe the 'Double-Rescaling' bug you diagnosed. How did it happen?

**Answer:** We set our Keras ImageDataGenerator to scale pixels to  $[0,1]$  using `rescale=1./255``. However, EfficientNetB0 has a built-in Rescaling layer inside the model that automatically normalizes inputs. This caused inputs to be scaled twice, shrinking the pixel ranges down to  $[0, 0.0039]$ .

### Q14: What were the training symptoms of the Double-Rescaling bug, and how did you trace it?

**Answer:** During training, validation loss remained flat and training accuracy did not improve. The model predicted only the majority class ('No Tumor') for every input. We realized the input pixel values arriving at the first conv layer were virtually zero, leading to vanishing gradients. Tracing the network layers revealed the double rescaling conflict.

### Q15: How did you resolve the Double-Rescaling bug and what was the impact?

**Answer:** We removed `rescale=1./255`` from our data generators, leaving pixel values as raw integers in the  $[0, 255]$  range. The model's built-in Rescaling layer then normalized it once. This fix restored the gradient flow, allowing the model to quickly converge to 99.59% accuracy on tumor categories and 99.99% on healthy scans.

**Q16: What is Grad-CAM and how does it benefit medical imaging?**

**Answer:** Grad-CAM (Gradient-weighted Class Activation Mapping) uses the gradients flowing into the final convolutional layer of the network to produce a coarse heatmap highlighting the crucial regions in the image. In medical computer vision, it explains the model's decisions, showing clinicians that the network is looking at the actual tumor area and not random noise or bones.

**Q17: Why did you target the 'top\_conv' layer of EfficientNetB0 for Grad-CAM?**

**Answer:** The final convolutional layer of a deep network contains the highest-level spatial features. It retains spatial information while representing abstract shapes and textures. Utilizing `top\_conv` provides the most localized and semantically meaningful activations for mapping tumor boundaries.

**Q18: Why did you convert the final model to ONNX format?**

**Answer:** ONNX (Open Neural Network Exchange) represents a unified format supported across frameworks. By converting Keras weights (.h5) to ONNX (.onnx), we decoupled the model from TensorFlow, which is very heavy, allowing us to use a lightweight runtime engine in production.

**Q19: What is the benefit of ONNX Runtime over TensorFlow in a web deployment?**

**Answer:** A TensorFlow installation requires about 2 GB of disk space and significant RAM, which is expensive and slow to spin up on free cloud environments. ONNX Runtime requires only ~15 MB of libraries, loads in seconds, and runs optimized model evaluations on standard CPUs in milliseconds, making deployment highly efficient.

**Q20: What is the user workflow of the deployed Hugging Face web application?**

**Answer:** The user visits the live space, uploads a brain MRI scan (.jpg/.png), and clicks 'Analyze'. The Gradio interface runs the image through the ONNX Runtime model on the backend, instantly returning a bar graph showing the confidence score across all four classes, accompanied by a short clinical description of the predicted category.